Modelos de difusión para IA generativa

Nota técnica 04

Modelo DDPM no condicional para predicción de ruido

Semillero GIDSYC | Universidad del Magdalena

Semillero GIDSYC | 2026-I

El objetivo de predicción de ruido deja de ser una expresión abstracta cuando se implementa una función entrenable $\epsilon_\theta(x_t, t)$. En este primer modelo, la red aprende a usar una imagen ruidosa y su tiempo de difusión para producir una estimación útil del ruido agregado.

1 Motivación

Las notas anteriores construyeron la secuencia conceptual que permite entrenar un modelo de difusión. Primero se formuló el problema generativo como aprendizaje de una distribución de datos. Después se introdujo el ruido gaussiano como distribución simple de referencia. Luego se definió el proceso directo de difusión y, finalmente, se obtuvo un objetivo supervisado de denoising:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right].$$

Esta nota estudia el primer paso entrenable. La función ϵ_θ ya no será una notación pendiente de implementación; será una red neuronal concreta. El problema no consiste en generar imágenes directamente con una red que reciba una semilla. Consiste en aprender un estimador de ruido que, usado de manera iterativa en el proceso inverso, permita transformar ruido gaussiano en muestras con estructura visual.

> **Idea clave 1 (Primer modelo entrenable).** El modelo no aprende $p_{\text{data}}(x)$ de forma explícita. Aprende una función de denoising condicionada por tiempo. La capacidad generativa aparece cuando esa función se inserta dentro de una cadena inversa.

2 Objeto de aprendizaje

El proceso directo permite construir, para cada dato limpio x_0, una versión ruidosa

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

El modelo recibe x_t y el índice temporal t. Su salida tiene la misma forma que la imagen de entrada:

$$\epsilon_\theta : \mathbb{R}^{1\times 28\times 28} \times \{0, \dots, T-1\} \longrightarrow \mathbb{R}^{1\times 28\times 28}.$$

En el notebook se usa la convención computacional de PyTorch: los índices temporales van de 0 a $T-1$. En la notación matemática de DDPM es común escribir los pasos como $1, \dots, T$. Ambas convenciones describen la misma cadena; solo cambia el desplazamiento del índice. En esta nota se conserva la notación x_t para facilitar la lectura de las ecuaciones, y se mencionará explícitamente cuando una expresión dependa de la convención computacional.

Para un mini-lote de tamaño B, la pérdida empírica usada en entrenamiento es

$$\widehat{\mathcal{L}}(\theta) = \frac{1}{B} \sum_{i=1}^{B} \left\| \epsilon^{(i)} - \epsilon_\theta(x_{t_i}^{(i)}, t_i) \right\|_2^2 .$$

Cada elemento del mini-lote usa un tiempo t_i y un ruido $\epsilon^{(i)}$ muestreados en el momento del entrenamiento. Por tanto, el dataset almacenado no contiene todos los pares posibles (x_t, ϵ). Solo contiene muestras limpias x_0; la supervisión se sintetiza dinámicamente.

> **Observación 1 (Datos almacenados y datos de entrenamiento).** El `DataLoader` carga imágenes limpias. El lote supervisado real se construye dentro del ciclo de entrenamiento al muestrear tiempos y ruido. Esta diferencia es importante: el número de ejemplos supervisados posibles es mucho mayor que el número de imágenes almacenadas.

3 Codificación temporal

El tiempo t no puede tratarse como un detalle auxiliar. Un mismo patrón visual puede significar cosas distintas dependiendo del nivel de ruido. En tiempos pequeños, x_t conserva mucha señal; en tiempos grandes, x_t está dominado por ruido. Por esa razón, el modelo debe recibir una representación explícita del tiempo.

En el notebook se usa una codificación sinusoidal. Para una dimensión par d_t, se define una familia de frecuencias ω_k y se construye

$$\gamma(t) = \left[\sin(t\omega_1), \dots, \sin(t\omega_{d_t/2}), \cos(t\omega_1), \dots, \cos(t\omega_{d_t/2}) \right].$$

Esta representación transforma el índice escalar t en un vector que puede ser procesado por capas lineales. En la arquitectura implementada, el embedding temporal se proyecta y se inyecta en los bloques convolucionales como un sesgo dependiente de t.

> **Idea clave 2 (Condicionamiento temporal).** La red no aprende una única operación de denoising. Aprende una familia de operaciones indexadas por el tiempo. El embedding temporal permite que la misma arquitectura adapte su predicción al nivel de ruido.

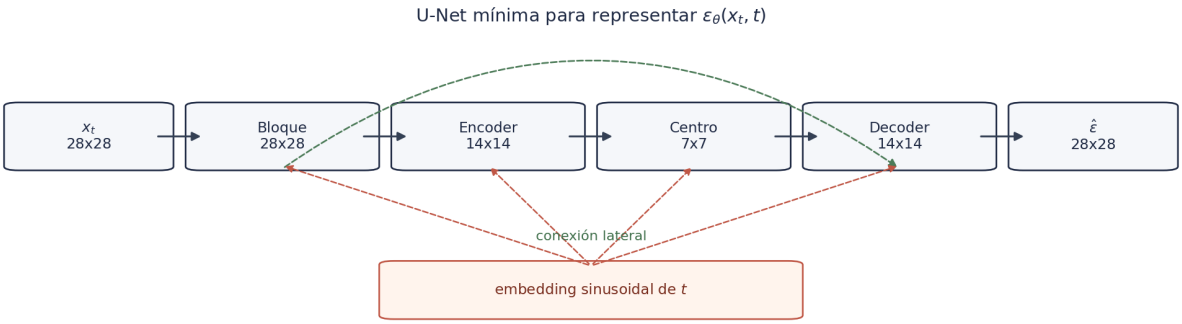
4 Arquitectura multiescala

Una CNN plana, que conserva la resolución 28×28 durante todo el procesamiento, puede aprender regularidades locales. Sin embargo, para generar imágenes completas se necesita coordinar información a escalas distintas. En QuickDraw, una casa no se reconoce únicamente por trazos locales; se reconoce por relaciones espaciales entre techo, paredes, base y puerta.

Por esa razón se usa una U-Net mínima. La arquitectura no busca competir con modelos modernos de difusión, sino proporcionar la menor estructura multiescala que haga visible la generación. La ruta principal reduce la resolución

$$28 \times 28 \longrightarrow 14 \times 14 \longrightarrow 7 \times 7 \longrightarrow 14 \times 14 \longrightarrow 28 \times 28.$$

Las conexiones laterales transportan información de alta resolución hacia la etapa de reconstrucción. La rama temporal alimenta los bloques convolucionales para que la predicción dependa de t.

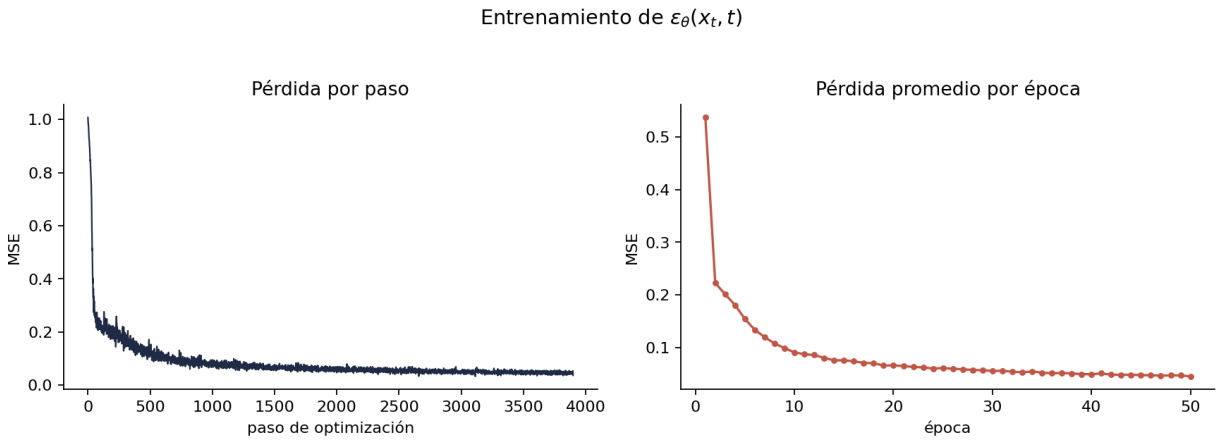


Formalmente, seguimos representando una función $\epsilon_\theta(x_t, t)$. La diferencia es que la familia funcional elegida tiene suficiente capacidad para combinar información local y global. En el experimento del notebook, la red tiene 969 953 parámetros entrenables, una escala razonable para un curso introductorio con imágenes pequeñas.

5 Entrenamiento

El ciclo de entrenamiento repite el siguiente procedimiento. Se toma un mini-lote de imágenes limpias x_0, se muestrea un vector de tiempos t, se muestrea ruido gaussiano ϵ, se construye x_t mediante la forma cerrada del proceso directo y se optimiza la pérdida cuadrática entre ϵ y $\epsilon_\theta(x_t, t)$.

En el experimento se usan imágenes de `house` de QuickDraw normalizadas a $[-1, 1]$, una agenda lineal de ruido con $T = 1000$, mini-lotes de tamaño 256, 50 épocas y el optimizador `AdamW`. En la GPU disponible durante el diseño del curso, el entrenamiento tarda alrededor de 100 segundos.



La curva por paso muestra variabilidad porque cada mini-lote contiene imágenes, tiempos y ruidos distintos. La curva por época revela la tendencia relevante: el error promedio decrece de forma clara. Esta disminución no prueba por sí sola que el modelo genere buenas muestras, pero sí indica que la tarea supervisada de predicción de ruido está siendo aprendida.

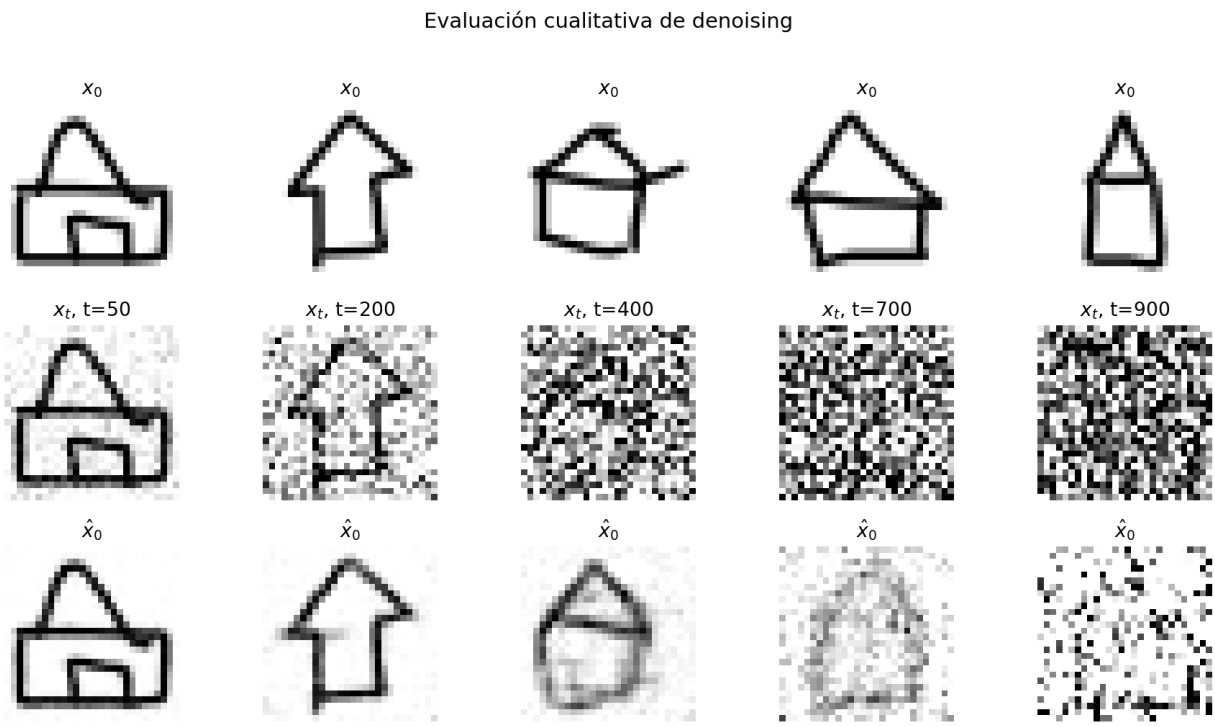
6 Denoising y reconstrucción

Una vez entrenado el modelo, podemos evaluar si su predicción de ruido permite reconstruir parcialmente una imagen limpia. A partir de la ecuación directa,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon,$$

se despeja la estimación

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\, \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$



La figura debe leerse con cuidado. No muestra generación desde cero. Muestra que, dada una imagen ruidosa x_t, la predicción del modelo contiene información suficiente para recuperar parte de la estructura de x_0, especialmente en tiempos intermedios. Para tiempos muy grandes, la reconstrucción es más difícil porque la señal original está fuertemente atenuada.

7 Primer muestreo inverso

El uso generativo aparece cuando la red se inserta en una cadena inversa, o cadena *reverse*. Se parte de

$$x_T \sim \mathcal{N}(0, I)$$

y se aplican pasos descendentes $T, T-1, \ldots, 1$. En una implementación DDPM básica, la media aproximada del paso inverso se escribe como

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

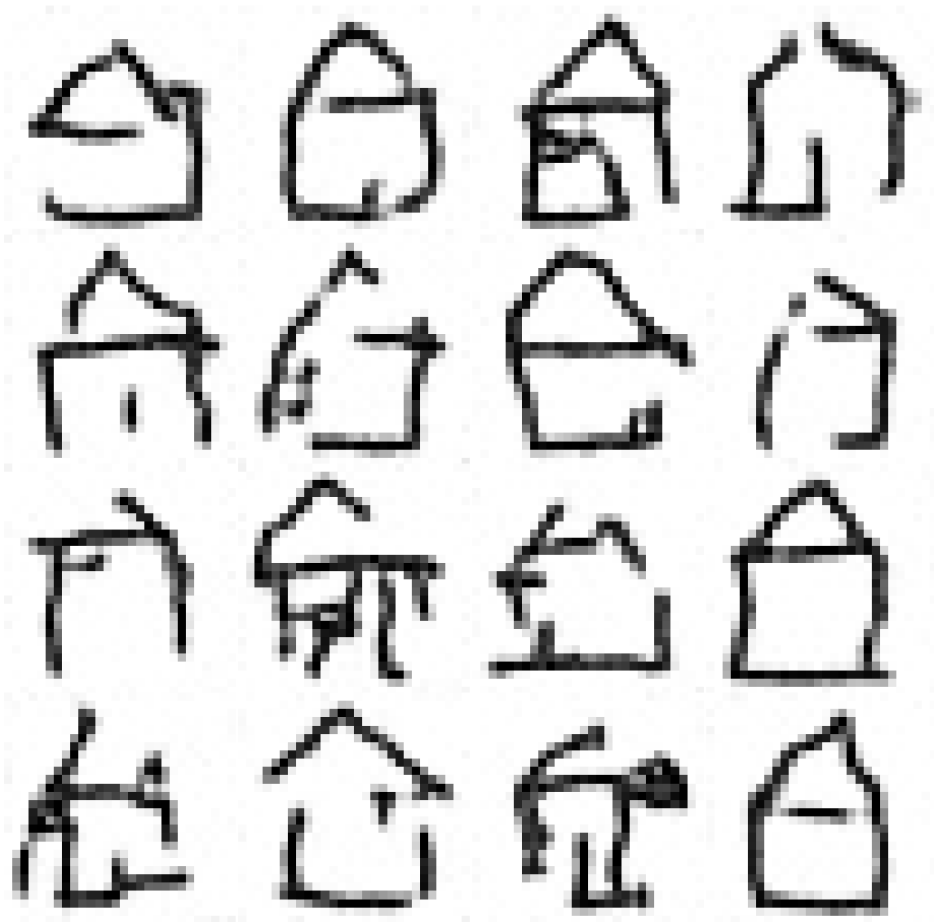
Luego se agrega ruido de acuerdo con una varianza posterior, salvo en el último paso. En el notebook se usa

$$\tilde{\beta}_t = \beta_t \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t},$$

de modo que, para $t > 1$,

$$x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\tilde{\beta}_t}\, z, \qquad z \sim \mathcal{N}(0, I).$$

Muestras generadas con el modelo entrenado



Estas muestras no deben interpretarse como el objetivo final del curso. Son una prueba funcional. Con una sola categoría, baja resolución y una arquitectura compacta, el procedimiento ya transforma ruido gaussiano en dibujos con estructura de casa. La calidad limitada es esperable: aún no hemos estudiado estrategias de muestreo, selección de pasos, estabilidad, comparación de semillas ni condicionamiento.

Observación 2 (Denoising no es lo mismo que muestreo). Evaluar $\hat{x}_0$ desde una imagen ruidosa comprueba una capacidad local del modelo. Muestrear desde ruido gaussiano exige que muchos pasos inversos funcionen de manera acumulativa. Por eso un modelo puede mostrar denoising razonable y aun así producir muestras imperfectas.

8 Limitaciones del experimento

El experimento fue diseñado para mostrar el mecanismo central con costo computacional razonable. Sus limitaciones son parte del aprendizaje:

1. El modelo es no condicional; solo aprende una distribución de casas, no una familia controlada por etiquetas.

2. Las imágenes son de 28×28, por lo que la evaluación visual debe ser cualitativa y moderada.
3. La arquitectura es mínima; no incluye atención, bloques residuales profundos ni técnicas modernas de estabilización.
4. El muestreo inverso se implementa de forma básica y todavía no se analiza sistemáticamente.
5. La pérdida MSE mide predicción de ruido, no calidad perceptual directa.

Estas restricciones no invalidan el resultado. Al contrario, delimitan con precisión qué se ha demostrado: el objetivo DDPM de predicción de ruido puede entrenarse y puede usarse generativamente en un escenario controlado.

9 Resumen

En esta nota se reemplazó la función abstracta $\epsilon_\theta(x_t, t)$ por una red neuronal concreta. La construcción mantiene la formulación matemática del notebook anterior, pero introduce los componentes necesarios para entrenar: mini-lotes dinámicos, embedding temporal, arquitectura convolucional multiescala, pérdida cuadrática y optimización por gradiente.

El resultado central es doble. Primero, el modelo aprende a predecir ruido en diferentes niveles de corrupción. Segundo, esa predicción puede insertarse en una cadena inversa para producir muestras desde ruido gaussiano. La generación observada no es perfecta, pero sí suficiente para cerrar el primer ciclo completo de un DDPM no condicional: difusión directa, entrenamiento de denoising y muestreo inverso básico.

10 Preguntas de estudio

1. ¿Por qué el modelo predice ϵ en lugar de predecir directamente una imagen final?
2. ¿Qué información aporta el tiempo t que no está contenida completamente en x_t?
3. ¿Por qué una arquitectura multiescala mejora el muestreo frente a una CNN plana?
4. ¿Qué diferencia conceptual hay entre la pérdida de entrenamiento y la calidad visual de las muestras?
5. ¿Por qué el dataset almacenado no necesita contener pares (x_t, ϵ)?
6. ¿Qué papel cumplen las conexiones laterales en una U-Net mínima?
7. ¿Por qué los errores pequeños de predicción pueden acumularse durante el muestreo inverso?
8. ¿Qué aspectos del muestreo deberían estudiarse antes de comparar modelos?

11 Continuidad

El siguiente notebook debe concentrarse en el muestreo. Ya tenemos un modelo entrenado que produce predicciones de ruido. Falta estudiar cómo usarlo mejor: número de pasos, semillas, trayectorias intermedias, estabilidad visual y criterios cualitativos de evaluación. Ese análisis permitirá separar dos preguntas distintas: qué tan bien se entrenó ϵ_θ y qué tan bien se está usando esa predicción para generar muestras.